Module C: temperature, and working backwards

What the newest part of LATTE does, why it refuses to answer some questions, and how to read what it gives back.

A note on symbols: textbooks write free energy with a capital Greek delta. This guide writes it dG, and a change in free energy is ddG, which is what the code calls them too. Energies are in kcal/mol, concentrations in nanomolar (nM), temperatures in kelvin (K). Room temperature, 25 degrees C, is 298.15 K. Body temperature, 37 degrees C, is 310.15 K.

1. The short version

LATTE already had a calculator that ran one way. You gave it numbers describing a biosensor and it told you how much brighter that sensor gets when it finds what it is looking for. That brightness ratio is called **fold change**. A fold change of 8 means the sample glows 8 times brighter with the target present than without it.

Module C adds two things on top.

- **Temperature.** The old calculator assumed a sensor behaves the same in a 4 degree C fridge as in a 37 degree C incubator. It does not. Module C redoes the chemistry at whatever temperature you actually run the experiment at, and when it does not have enough information to do that honestly, it says so rather than guessing.
- **Working backwards.** Instead of asking "what fold change will this design give me", you can now ask "I need a fold change of 8, which designs get me there". That is the harder question, and it is the one a lab actually has.

2. The sensor, in one paragraph

The sensor is a protein that works like a lock. A **cage** sits shut most of the time, held that way by an arm called the **latch**. A separate piece called the **key** makes light, but only once the cage is open. When the **target** (a virus protein, say) binds, it pulls the latch off, the cage opens, the key gets in, and the sample lights up. Fold change is just the light with target divided by the light without it.

How tightly each piece holds on is written as an energy. More negative means a tighter grip. These are the five numbers Module C can solve for.

Parameter	What it means in words	Typical value
dG_latch_cage	How hard the latch grips the cage. Very negative means a cage that stays shut, which gives a dark background but is harder to open.	-4 kcal/mol
dG_key_cage	How hard the light-making key grips the open cage.	-11 kcal/mol
dG_target_binder	How hard the target grips the part built to catch it.	-12 kcal/mol
key_conc	How much key you pipette in.	500 nM

Parameter	What it means in words	Typical value
target_conc	How much target is in the sample. This is usually what you are trying to measure, not something you choose.	0.1 to 100 nM

One more number matters and cannot be solved for: **pull**, how strongly target binding yanks on the latch. It is set by the shape of the cage and latch, so changing it means redesigning the protein, not changing a concentration.

3. Part 1: temperature

An energy is not really one number. It is made of a heat part (dH, enthalpy) and a disorder part (dS, entropy), and those two trade off differently as things warm up. The equation that describes the trade is the Gibbs-Helmholtz equation, which Module C implements in full:

```
dG(T) = dH_ref - T*dS_ref + dCp*[(T - T_ref) - T*ln(T/T_ref)]
```

Read it left to right: start from what was measured at a reference temperature, correct for the new temperature, then apply a second correction using dCp (heat capacity). That second term is small a few degrees away from the reference and gets important further out, because proteins change how much heat they soak up as they fold and unfold.

Here is the catch, and it is the whole reason this part of the module exists. You can only run that equation if somebody measured dH. Plenty of published values are just a dG at one temperature, and from one number there is no honest way to extrapolate. So Module C sorts whatever you give it into one of three modes and stamps every answer with the mode it used, so nobody has to guess how much to trust it.

Mode	What you supplied	What it does	How far to trust it
full	dH and dCp	The whole equation above.	Good across a wide range.
vant_hoff	dH only	Same equation with dCp taken as zero.	Fine within about 15 K of the reference. Warns past that.
isothermal	dG only	Nothing. Returns the reference value unchanged.	It is not a value at your temperature at all, and it says so.

That bottom row is the point. Ask for 37 degrees C with only a room-temperature number and you get the number you started with, plus this:

> *"TEMPERATURE SCALING UNAVAILABLE for dG_latch_cage: only a reference dG at 298.15 K was supplied, so the value at 310.00 K is the UNSCALED 298.15 K value. Supply dH_ref (and ideally dCp) to extrapolate. Do not read the fold change as a prediction at 310.00 K."*

It would have been easy to quietly return something that looks like an answer. A number that looks fine and is wrong is worse than no number, because it ends up in a poster.

Two smaller pieces come with this:

- **A dCp estimator.** If you know roughly how much surface area gets buried when the two pieces stick together, there is a helper that turns that into a dCp. It is an estimate from a published rule of thumb, not

a measurement, and any answer that used it carries a line saying exactly that.

- **A melting warning.** Give it a melting temperature (Tm) and it warns when your working temperature comes within 10 K of it. Past that point the cage is falling apart on its own, so the whole open-or-shut picture stops describing reality and the fold change is meaningless rather than merely imprecise.

All three of the older fold-change endpoints now take an optional temperature_K, and it defaults to 298.15 K. At the default they return exactly what they returned before, to the last digit. That is checked by a test that compares the two responses field by field.

4. Part 2: working backwards

How much can honestly be said depends entirely on how many parameters you let float. So there are three different answers rather than one, and the module picks based on the length of your solve_for list.

One free parameter

There is a single answer and a well-known method (Brent's) finds it quickly. Before trusting that, the solver checks whether fold change moves in one consistent direction across your search range. If it ever doubles back, reporting one answer would be hiding the others, so it reports every solution it can bracket on a fine scan.

Two free parameters

Now there is no single answer. There is a whole curve of combinations that all hit your target, and the curve is the useful output, so you get it as a list of coordinates ready to plot. You also get one recommended point picked off that curve: the one sitting furthest from the edge of the acceptable region.

The reason is practical. Think of the acceptable designs as a road. Two designs can both be on the road, but one is in the middle of the lane and one is half off the shoulder. Pipetting is never exact and neither is a measured energy, so the middle-of-the-road design is the one that still works tomorrow. That is what "most robust" means here, and it is measured as distance to the edge in each parameter's own natural units.

Three or more free parameters

The region becomes a shape in three or more dimensions. It cannot be drawn, and mapping it properly costs far more than it is worth, so the module does not pretend to. Instead you get the recommended design, and for each parameter separately, how far that one can drift on its own before the design misses the spec. That per-parameter range is the number you can actually hand to someone at a bench.

When the target is impossible

This case matters more than it sounds. The chemistry has a hard ceiling: at a pull strength of 10, fold change cannot get much past about 11, no matter what you do to the energies or the concentrations. Ask for 1000 and the module does not hand back its closest guess and hope you do not notice. It reports that the target is out of reach, the best it could actually do, where that happens, and which parameter is the thing blocking it:

```
feasible: false
message: Target fold change 1000 is not reachable for any dG_latch_cage in [-10, -1].
         Closest is 11.0 at dG_latch_cage = -10.
binding_constraint: dG_latch_cage at its lower bound (-10)
```

Read that as: stop tuning the latch, it is already as tight as you allowed, and it is not the thing standing in your way. Raise the pull, which means redesigning the cage and latch geometry.

5. What comes back with every answer

Field	What it tells you
values	The parameter values that hit your target, in kcal/mol and nM.
fold_change	What those values actually achieve. Compare it to what you asked for.
sensitivity	How much the fold change moves when a parameter moves a little. A big number means a touchy design: a small mistake in that parameter shows up as a big change in brightness.
tolerance	The range each parameter can wander over on its own while the design still meets spec. Narrow means fussy.
warnings	Everything the temperature layer wanted to tell you, including the refusal to extrapolate and the melting-point warning.
thermo_mode	full, vant_hoff or isothermal. If several energies are involved, the least trustworthy one is reported, because that is the one that limits the answer.
feasible	false means the target was out of reach and you should read closest instead of looking for a solution.

6. Part 3: turning an energy into DNA you can order

The solver answers in kcal/mol. It will tell you that you need dG_latch_cage to be -5.5, which nobody can order. A lab orders DNA coding for specific amino acids, so something has to bridge the gap.

You hand this part your current latch sequence, its current energy, and which positions sit at the interface. It works out the gap it needs to close (the required ddG), then tries swapping residues at those positions and scores each swap using the same sequence rules Module A already uses: acidic residues (D and E) at an interface cost you binding, and residues that form good helices help hold the latch together. Single swaps and pairs both get tried, and you get the closest ones back, written the way a lab writes them. D4N means the D at position 4 becomes an N.

Each candidate carries a confidence flag, and the flags are deliberately modest:

- **moderate**: the charge rule is doing most of the work, and that rule is calibrated against a real sequence pair from this project (PfLB-1 and its optimized version).
- **low**: the helix part is doing most of the work, which is a scaling guess rather than a measurement. Every double mutant is also low, because two changes almost never add up as neatly as the arithmetic pretends.

The response states in plain words that these are heuristic rankings, not predictions, and that they need checking with Rosetta or FoldX or a plate reader before anyone orders a big panel of DNA. Ranking is deliberately coarse, to 0.1 kcal/mol, because the estimate is not better than that. Anything inside one bin counts as a tie, and a single mutation beats a double in a tie, since one change is cheaper to order and likelier to behave.

7. How it was checked

88 new tests, and the suite now stands at 282 passing. Three tests fail, and the same three were already failing before any of this was written, in unrelated sequence-checking code.

The central test is a round trip. Pick random but valid parameters, run the old calculator forward to get a fold change, hand that fold change to the new solver, and check that the design it comes back with produces the same fold change. If the two halves ever disagree, one of them is wrong, and the test says which parameter set broke it.

The temperature equation is checked against values worked out by hand at four temperatures a lab actually uses, using dH = -12.0 kcal/mol, dS = -0.025 kcal/mol/K and dCp = -0.5 kcal/mol/K:

Temperature	Why that temperature	dG(T), hand-computed
277.00 K	4 degrees C, a cold room or a fridge	-4.6907 kcal/mol
298.15 K	25 degrees C, the reference point	-4.5463 kcal/mol
310.00 K	about 37 degrees C, body temperature	-4.1338 kcal/mol
328.00 K	55 degrees C, hot enough to start worrying about the cage	-3.0766 kcal/mol

The rest of the suite covers the things most likely to go quietly wrong: unreachable targets take the reporting path instead of raising an error or inventing a number, isothermal mode refuses to extrapolate, non-monotonic cases return every root, and 298.15 K reproduces the old outputs exactly.

8. What it deliberately does not do

- No molecular dynamics, no structure prediction, no GPU. Everything runs on a laptop CPU, and a full inverse design finishes in well under a tenth of a second.
- No screen for it yet. Module C is API only, at POST /api/inverse_design and POST /api/mutations. The numbers coming back are already in the units the interface uses, so a tab can be added without touching the maths.
- It does not invent thermodynamic data. If you do not have dH, it will tell you what it cannot do rather than filling the gap with a plausible guess.